Universidad Nacional de Costa Rica

Escuela de Informática

EIF205 Arquitectura de Computadoras

Proyecto II: Tetris en ensamblador MIPS

Informe técnico

Integrantes:

Keilor Baltodano

Andrey Chacón Sánchez

Jorge Navarro Vega

Docente: César Valverde Canossa

8 de noviembre de 2026

1. Introducción

El presente informe documenta el desarrollo del Proyecto II del curso EIF205 Arquitectura de Computadoras, cuyo objetivo fue construir un juego 2D en lenguaje ensamblador con código modular, uso correcto de la pila y de los registros, depuración documentada y trabajo colaborativo mediante Git. El equipo eligió el juego Tetris y lo implementó para el simulador MARS 4.5 (arquitectura MIPS32), empleando el Bitmap Display como salida gráfica y el Keyboard and Display MMIO Simulator como entrada.

Con autorización del profesor, el trabajo partió de una implementación pública existente (repositorio tetris_mips_asm), contenida en un único archivo de aproximadamente 1680 líneas. La labor del equipo consistió en reorganizar esa base en módulos, corregir sus errores de manejo de registros y completar las funciones que exige el enunciado: estados explícitos, niveles, puntuación clásica, nuevas mecánicas e interfaz en español. El informe sigue la estructura solicitada en el enunciado: arquitectura del software y mapeo de memoria, descripción de módulos y rutinas, bitácora de pruebas y conclusiones.

1.1 Base de partida y aportes del equipo

La Tabla 1 resume las diferencias entre la versión base y la versión final. La división del trabajo asignó a Keilor el núcleo (estructura, estados, teclado, tiempo y niveles), a Andrey la lógica (piezas, colisiones, líneas y puntaje) y a Jorge los gráficos y la presentación.

Tabla 1. Comparación entre la versión base y la versión final

Aspecto	Versión base	Versión final
Estructura	Un solo archivo; sin macros ni constantes simbólicas.	Ocho archivos unidos con .include; constantes .eqv y macros de pila y dibujo.
Estados	Solo una bandera GAME_OVER_FLAG.	Máquina de estados MENÚ, JUGANDO, PAUSA, GAME OVER y VICTORIA.
Tablero	16 × 32 celdas con tamaños escritos a mano.	10 × 20 celdas definidas por ANCHO y ALTO.
Niveles	Aceleración de 300 ms por línea hasta 100 ms.	10 niveles con tabla de velocidades; un nivel cada 5 líneas; victoria al completar el nivel 10.
Puntaje	1 punto por línea.	100, 300, 500 u 800 puntos por 1 a 4 líneas, multiplicados por el nivel.

Aspecto	Versión base	Versión final
Mecánicas	Rotación cancelada junto a la pared; sin caída instantánea.	Rotación con corrimiento (wall kick) y caída instantánea con la barra espaciadora.
Registros	Uso de $k0/$k1 y de $s0–$s1 sin preservarlos.	Registros del kernel eliminados; marcos de pila revisados en todos los módulos.
Interfaz	Redibujo completo cada ciclo, panel "siguiente" vacío, un solo color para bloques fijos, textos en inglés.	Doble búfer sin parpadeo, vista previa de la pieza siguiente, color por pieza, nivel y líneas en pantalla, textos en español.

2. Arquitectura del software y mapeo de memoria

2.1 Organización modular

El programa se compone de ocho archivos que MARS ensambla como una sola unidad a partir de main.asm. Primero se incluyen las macros y los archivos de datos, para que las constantes .eqv y las macros estén definidas antes de usarse; al final se incluyen los archivos con procedimientos, de modo que la ejecución comience en la etiqueta main. Cada módulo utiliza un prefijo propio en sus etiquetas (nuc_, log_ y gfx_) para evitar colisiones de nombres. La Figura 1 muestra los módulos y sus dependencias; su contenido se describe en la sección 3.

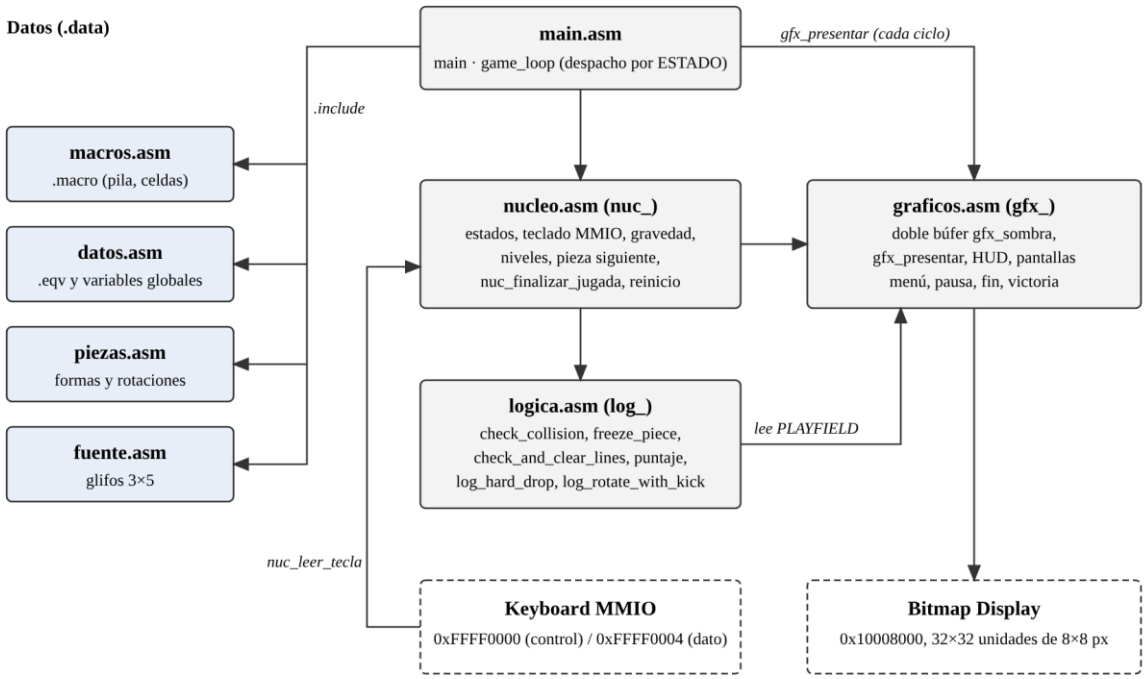


Figura 1. Diagrama de arquitectura del programa

2.2 Ciclo principal y máquina de estados

La variable ESTADO, almacenada en datos.asm, determina el comportamiento del programa. En cada iteración, game_loop lee su valor y llama al procedimiento correspondiente (nuc_estado_menu, nuc_estado_jugando, nuc_estado_pausa, nuc_estado_gameover o nuc_estado_victoria). Cada procedimiento realiza una sola pasada y retorna; después, el ciclo presenta el cuadro en pantalla y espera 100 ms mediante la llamada al sistema 32. Este diseño evita bucles de espera dentro de los estados y mantiene un único punto de control del flujo. La Figura 2 muestra las transiciones; la tecla Q termina el programa desde cualquier estado.

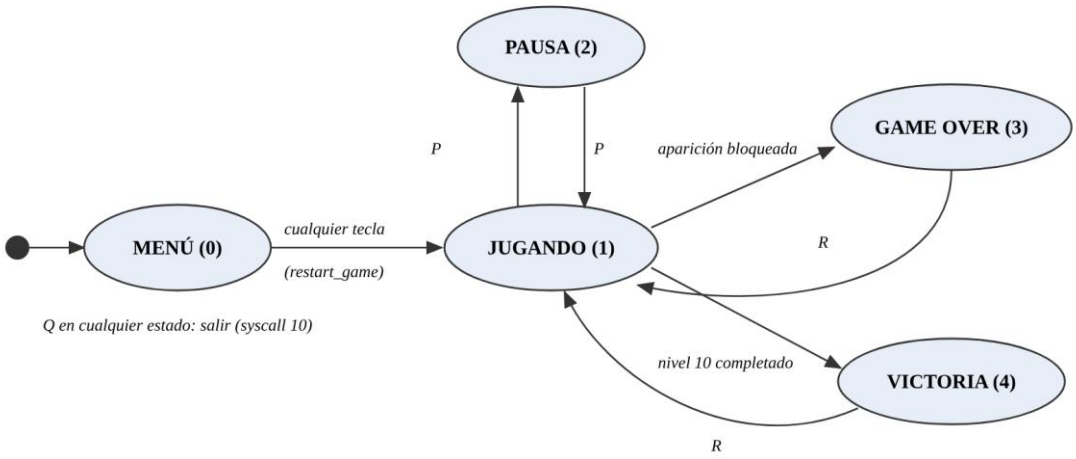


Figura 2. Máquina de estados del juego

2.3 Convenciones de registros y pila

El equipo adoptó la convención de MIPS: $a0–$a3 para argumentos, $v0–$v1 para resultados, $t0–$t9 como temporales no preservados y $s0–$s7 como registros que el procedimiento llamado debe guardar y restaurar. Todo procedimiento que llama a otro guarda $ra en la pila, y $sp se restaura antes de retornar. Los procedimientos hoja, como nuc_leer_tecla o log_update_score, no reservan pila. Se eliminó el uso de $k0 y $k1, reservados al kernel, que la base empleaba para leer el teclado. Las macros guardar_ra_s0 … restaurar_ra_s0_s1_s2_s3_s4 de macros.asm uniformizan los marcos de pila.

2.4 Mapeo de memoria

El programa utiliza cuatro regiones de memoria. Las direcciones del segmento de datos se obtuvieron siguiendo el orden de ensamblado y se verificaron contra la vista Data Segment de MARS durante la ejecución (Tablas 2 y 3).

Tabla 2. Regiones de memoria

Región	Direcciones	Uso
Segmento de texto	0x00400000 en adelante	Instrucciones del programa; main en 0x00400000.
Segmento de datos (.data)	0x10010000 – 0x10011BDB	Variables globales, tablas de piezas, fuente y búfer de dibujo (7132 bytes).
Bitmap Display	0x10008000 – 0x10008FFF	1024 palabras (32 × 32 unidades de 8 × 8 píxeles), color 0x00RRGGBB.
Keyboard MMIO	0xFFFF0000 / 0xFFFF0004	Registro de control (1 si hay tecla nueva) y registro de dato (código ASCII).
Pila	Desde 0x7FFFEFFC hacia abajo	Marcos de los procedimientos ($ra y registros $s).

Tabla 3. Variables principales del segmento de datos

Dirección	Etiqueta	Tamaño	Descripción
0x10010000	ADDR_DSPL	4 B	Dirección base del Bitmap Display.
0x10010004	ADDR_KBRD	4 B	Dirección del Keyboard MMIO.
0x10010008	COLOR_BLACK … COLOR_TEXT_ORANGE	60 B	Colores de fondo, paneles, piezas y texto.
0x10010044	CURRENT_PIECE_X / _Y	8 B	Posición de la pieza activa.
0x1001004C	CURRENT_PIECE_TYPE / _ORIENTATION / _COLOR	12 B	Tipo (0–6), orientación (0–3) y color de la pieza activa.
0x10010058	SCORE	4 B	Puntaje de la partida.
0x1001005C	LINES_CLEARED	4 B	Total de líneas borradas.
0x10010060	LAST_GRAVITY_TIME	4 B	Tiempo de la última caída por gravedad.
0x10010064	GRAVITY_INTERVAL	4 B	Intervalo de caída del nivel actual (ms).
0x10010068	NIVEL	4 B	Nivel actual (1–10).
0x1001006C	SIGUIENTE_PIEZA	4 B	Tipo de la próxima pieza; −1 antes del primer sorteo.
0x10010070	TABLA_VELOCIDAD	40 B	Intervalos de 1000 a 120 ms para los niveles 1 a 10.
0x10010098	HIGH_SCORE	4 B	Récord de la sesión.
0x1001009C	ESTADO	4 B	Estado actual (0–4).

Dirección	Etiqueta	Tamaño	Descripción
0x100100B8	PLAYFIELD	800 B	Tablero 10 × 20; cada celda guarda 0 (vacía) o tipo + 1.
0x100103EC	MASTER_SHAPE_TABLE	28 B	Punteros a las tablas de rotación de cada pieza.
0x10010408	ALL_COLORS_TABLE	28 B	Punteros al color de cada tipo de pieza.
0x10010424	ROTATIONS_* / SHAPE_*	720 B	Tablas de rotación y desplazamientos (dx, dy) de cada forma.
0x100106F4	Glifos gfx_g_*	860 B	Mapas de bits de la fuente.
0x10010A50	gfx_fuente_tabla	236 B	Tabla de búsqueda ASCII 32–90.
0x10010B3C	gfx_sombra	4096 B	Búfer de dibujo (copia en memoria de la pantalla).
0x10011B3C	gfx_cuadros_completos	4 B	Contador de cuadros iniciales copiados por completo.

3. Descripción de módulos y rutinas

3.1 Módulos

La Tabla 4 describe el contenido de cada archivo y su responsable. Los archivos de datos (datos.asm, piezas.asm y fuente.asm) solo contienen la sección .data; los demás contienen procedimientos.

Tabla 4. Módulos del programa

Archivo	Responsable	Contenido principal
main.asm	Keilor	Punto de entrada, inclusiones y bucle principal game_loop, que despacha según ESTADO y llama a gfx_presentar cada 100 ms.
datos.asm	Keilor	Constantes (ANCHO, ALTO, NIVEL_MAX, LINEAS_POR_NIVEL, teclas y estados) y variables globales: tablero, pieza actual, puntaje, nivel, pieza siguiente y tabla de velocidades.
nucleo.asm	Keilor	Lectura del teclado (nuc_leer_tecla), procedimientos de cada estado, entrada del jugador, gravedad por tiempo, generación de piezas, niveles (update_speed), cierre de jugada y reinicio.
logica.asm	Andrey	Colisiones, fijación de piezas, borrado y desplazamiento de filas, puntaje (log_update_score), caída instantánea (log_hard_drop) y rotación con corrimiento (log_rotate_with_kick).

Archivo	Responsable	Contenido principal
piezas.asm	Andrey	Tablas de las siete piezas con sus cuatro orientaciones y tabla de colores por tipo.
macros.asm	Jorge	Macros para guardar y restaurar $ra y $s0–$s4, calcular la dirección de una celda, pintar una celda y leer o escribir variables globales.
graficos.asm	Jorge	Doble búfer de pantalla, fondo, tablero, piezas, vista previa, HUD (puntaje, récord, nivel y líneas) y pantallas de menú, pausa, fin del juego y victoria.
fuente.asm	Jorge	Mapas de bits de una fuente de 3 × 5 unidades para letras, dígitos y signos.

3.2 Rutinas principales

La Tabla 5 resume las rutinas que forman las interfaces entre módulos. Cada una documenta en su encabezado los argumentos, el valor de retorno y los registros que preserva, lo que permitió que cada integrante desarrollara y probara su módulo de forma independiente.

Tabla 5. Rutinas principales y sus interfaces

Rutina	Módulo	Entrada / salida	Función
nuc_leer_tecla	nucleo	Salida: $v0 = código ASCII o 0.	Consulta el registro de control del Keyboard MMIO y lee la tecla si hay una nueva.
nuc_estado_jugando	nucleo	—	Dibuja la escena, procesa la entrada (handle_input) y aplica la gravedad (apply_gravity).
nuc_finalizar_jugada	nucleo	—	Cierra una jugada: líneas, puntaje, nivel y pieza nueva.
spawn_new_piece	nucleo	—	Toma la pieza siguiente, sortea otra y detecta el fin del juego.
update_speed	nucleo	—	Calcula el nivel, actualiza GRAVITY_INTERVAL o declara la victoria.
restart_game	nucleo	—	Restablece tablero, puntaje, nivel y estado para una partida nueva.
check_collision	logica	$a0 = orientación, $a1 = X, $a2 = Y; $v0 = 1 si choca.	Verifica límites del tablero y celdas ocupadas para una posición candidata.

Rutina	Módulo	Entrada / salida	Función
freeze_piece	logica	$v0 = 1 si se rechaza.	Escribe la pieza activa en PLAYFIELD con el valor tipo + 1.
check_and_clear_lines	logica	$v0 = líneas borradas.	Elimina filas completas con shift_rows_down y actualiza LINES_CLEARED.
log_update_score	logica	$a0 = líneas, $a1 = nivel.	Suma 100, 300, 500 u 800 puntos multiplicados por el nivel.
log_hard_drop	logica	$v1 = 0 si la pieza se fijó.	Baja la pieza hasta la última posición válida y la fija.
log_rotate_with_kick	logica	$v0 = 1 si rotó.	Intenta rotar en X, X − 1 y X + 1; si todo falla, cancela.
draw_scene	graficos	—	Dibuja fondo, tablero y bloques fijados en el búfer de dibujo.
draw_tetromino	graficos	Pieza activa (globales).	Dibuja la pieza que cae con su color.
gfx_dibujar_siguiente	graficos	$a0 = tipo de pieza.	Dibuja la vista previa de la pieza siguiente.
draw_string / draw_score	graficos	$a0 = texto, $a1 = x, $a2 = y.	Escriben textos y números con la fuente de 3 × 5.
gfx_pantalla_mensaje	graficos	$a0 = tabla de textos.	Base de las pantallas de menú, pausa, fin del juego y victoria.
gfx_presentar	graficos	—	Copia al Bitmap Display solo las celdas que cambiaron (doble búfer).

3.3 Flujo de una jugada

Durante el estado JUGANDO, el núcleo dibuja la escena, procesa la tecla recibida y aplica la gravedad cuando transcurre el intervalo del nivel actual, medido con la llamada al sistema 30. Una pieza puede fijarse de tres formas: con la tecla S, por gravedad o con la caída instantánea. En los tres casos se llama una sola vez al procedimiento nuc_finalizar_jugada, que ejecuta en orden las siguientes acciones:

- `check_and_clear_lines` elimina las filas completas y devuelve cuántas se borraron.

- `log_update_score` suma el puntaje correspondiente según el nivel vigente antes de un posible ascenso.
- `update_speed` recalcula el nivel como líneas / 5 + 1, actualiza la velocidad desde TABLA_VELOCIDAD o cambia ESTADO a VICTORIA al completar el nivel 10.
- Si el jugador no ha ganado, reinicia el temporizador de gravedad y llama a `spawn_new_piece`, que toma SIGUIENTE_PIEZA como pieza actual, sortea la siguiente con la llamada al sistema 42 y pasa a GAME OVER si la posición de aparición está ocupada.

Centralizar este cierre evita que una jugada se puntúe dos veces y garantiza que las tres vías de fijación produzcan el mismo resultado.

4. Bitácora de pruebas

Las pruebas se ejecutaron en MARS 4.5 con el Bitmap Display configurado en unidades de 8 × 8, pantalla de 256 × 256 y base 0x10008000, y con el Keyboard MMIO conectado. Se organizaron en tres grupos, según lo solicitado: colisiones y límites, estados del juego y registro de errores.

4.1 Casos de prueba de colisiones y límites

Las pruebas de la lógica se automatizaron con tres programas independientes (andrey/pruebas/etapa1.asm a etapa3.asm) que llaman a los procedimientos reales de logica.asm sobre un tablero rodeado de valores centinela, de modo que cualquier escritura fuera del tablero se detecta. Antes de cada llamada se cargan $s0–$s7 con valores conocidos para comprobar que se preservan. En total se ejecutaron 235 escenarios con 37 911 comprobaciones, todas aprobadas. La Tabla 6 resume los casos representativos; las capturas y registros de cada uno se encuentran en andrey/evidencias/capturas.

Tabla 6. Casos representativos de colisiones y límites

Caso	Situación probada	Resultado observado	Evidencia
P02	Pieza O en X = 0 y X = 8 (válidas) y en X = −1 y X = 9 (cruzan paredes).	check_collision devuelve 0 en las válidas y 1 en las que cruzan; tablero intacto.	P02_pared_izquierda, P02_pared_derecha
P03	Pieza apoyada en el piso, fila 19 y filas negativas.	Ningún bloque queda bajo la fila 19 ni sobre la fila 0; una I vertical con X = −1 es válida si sus bloques quedan dentro.	P03_piso, P03_limite_superior

Caso	Situación probada	Resultado observado	Evidencia
P04	Bloques fijados de cada tipo (ID 1–7) y aparición ocupada.	Cualquier ID distinto de cero bloquea; la pieza rechazada no cambia.	P04_aparicion_bloqueada
P05	Fijación con un bloque fuera del tablero.	freeze_piece devuelve 1 sin escrituras parciales; centinelas intactos.	P05_fijacion_rechazada
P06	Rotación de la T junto a la pared derecha e izquierda.	X pasa de 8 a 7 y de −1 a 0; la pieza rota sin salir del tablero.	P06_kick_izquierda, P06_kick_derecha
P07	Rotación con obstáculos en las tres posiciones posibles.	La rotación se cancela y la pieza conserva posición y orientación.	P07_rotacion_cancelada
P08	Caída instantánea con obstáculos intermedios.	La pieza se detiene sobre el primer obstáculo (Y = 3) y no lo atraviesa.	P08_caida_obstaculo
P10	Una, dos, tres y cuatro líneas completas.	Devuelve 1 a 4; el marcador superior baja esa cantidad de filas.	P10_una_linea … P10_cuatro_lineas
P11	Desplazamiento con filas −1, 20, 0, 1 y 19.	Las filas inválidas no cambian nada; las válidas conservan los IDs.	P11_desplazamiento_ids
P15	Registros, pila y centinelas tras cada llamada.	$s0–$s7 y $sp iguales a los valores iniciales en todos los casos.	P15_registros_pila

La Figura 3 muestra la evidencia del caso P06: tras log_rotate_with_kick, la pieza T que estaba en X = −1 queda en X = 0 con la orientación siguiente, $v0 = 1 indica rotación exitosa, los registros $s0–$s7 conservan los valores 1000 a 1007 y los centinelas mantienen el valor 123456789.

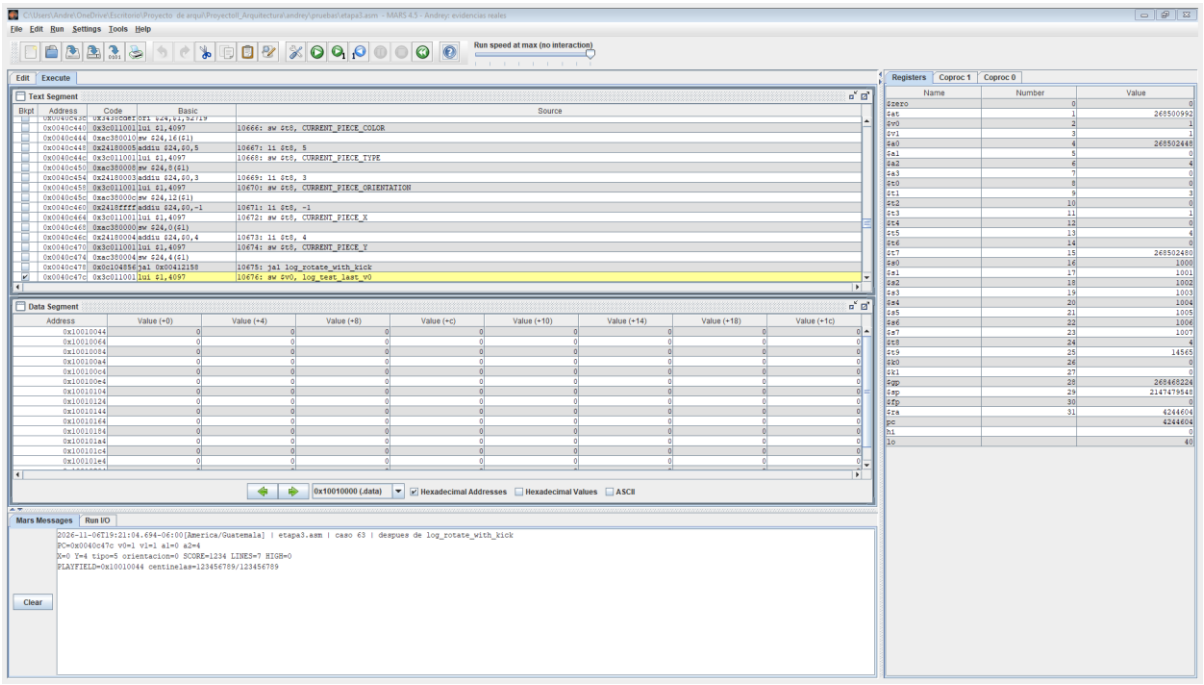


Figura 3. Evidencia del caso P06 (rotación con corrimiento junto a la pared)

4.2 Pruebas de estado

Las pruebas de estado verificaron el flujo entre el menú principal, la partida, la pausa, el fin del juego, la victoria y el reinicio. La prueba de victoria se realizó reduciendo temporalmente NIVEL_MAX y LINEAS_POR_NIVEL a 1, ya que ambos valores son constantes. Todos los casos de la Tabla 7 produjeron el resultado esperado.

Tabla 7. Casos de prueba de estado

Caso	Estado y acción	Resultado esperado	Res.
E-01	Inicio del programa	Pantalla de menú con “TETRIS” y “PRESIONA UNA TECLA”.	Correcto
E-02	MENÚ: cualquier tecla	Partida nueva: tablero vacío, PTS 0, NV 1, LN 0 y pieza siguiente visible.	Correcto
E-03	JUGANDO: tecla P	Pantalla “PAUSA (P)”; la pieza deja de caer.	Correcto
E-04	PAUSA: tecla P	Se reanuda la partida sin caída inmediata (temporizador reiniciado).	Correcto
E-05	JUGANDO: completar una línea	La línea desaparece; PTS aumenta 100 y LN aumenta 1.	Correcto
E-06	JUGANDO: barra espaciadora	La pieza cae hasta el fondo y se fija con su color.	Correcto
E-07	JUGANDO: W junto a una pared	La pieza rota desplazándose una columna (wall kick).	Correcto

Caso	Estado y acción	Resultado esperado	Res.
E-08	JUGANDO: aparición bloqueada	Pantalla “FIN DEL JUEGO (R)” y récord actualizado.	Correcto
E-09	GAME OVER: tecla R	Partida nueva con puntaje 0 y nivel 1.	Correcto
E-10	JUGANDO: completar el último nivel	Pantalla “VICTORIA”; no aparece una pieza nueva.	Correcto
E-11	VICTORIA: tecla R	Partida nueva desde el nivel 1.	Correcto
E-12	Cualquier estado: tecla Q	El programa termina con la llamada al sistema 10.	Correcto

4.3 Prueba integrada

Después de integrar los tres módulos se jugó una partida completa en MARS. Se verificó la aparición del menú, el inicio de la partida, el movimiento, la rotación con corrimiento, la caída instantánea, el color de cada pieza fijada, la vista previa de la siguiente pieza, la pausa, el borrado de una línea con 100 puntos, la pantalla de fin del juego y el reinicio. En esta prueba se detectó que el menú inicial no se mostraba, error que se corrigió y se registra como G-01 en la Tabla 8.

4.4 Registro de errores encontrados y soluciones

La Tabla 8 resume los errores detectados durante el desarrollo, el impacto que causaron y la solución aplicada. Los identificadores N e INT corresponden al núcleo y a la integración, LOG y PRU a la lógica, XJ y G a los gráficos, y E a limitaciones del simulador. Todos los errores quedaron resueltos; las limitaciones E-02 y E-03 se mitigaron mediante el procedimiento de uso indicado.

Tabla 8. Registro de errores, impacto y solución

ID	Error detectado	Impacto	Solución aplicada
N-01	Etiqueta nuc_prueba definida dos veces al crear el esqueleto de módulos.	MARS rechazaba el ensamblado; el programa no podía ejecutarse.	Se eliminó la definición duplicada. Ensambla sin errores.
N-02	Desalineación de memoria: PLAYFIELD quedó en 0x1001008F tras cadenas .asciiz.	Excepción “store address not aligned” en el primer sw; el juego se detenía al iniciar.	Directiva .align 2 antes de PLAYFIELD y al inicio de piezas.asm y fuente.asm.

ID	Error detectado	Impacto	Solución aplicada
N-03	Tramo de game_loop perdido al copiar la base a los módulos.	El programa terminaba con “dropped off bottom” sin llegar a jugarse.	Se restauró el tramo faltante desde la base y se comparó el comportamiento.
N-04	gl_jugando no llamaba a nuc_estado_jugando.	El estado JUGANDO no hacía nada: la pieza no caía ni respondía.	Se agregó la llamada. La partida avanza normalmente.
N-05	Instrucción sw duplicada en spawn_new_piece al agregar SIGUIENTE_PIEZA.	La pieza que caía no coincidía con la vista previa.	Se eliminó la línea repetida y se verificó la coincidencia.
INT-03	El núcleo no llamaba al puntaje nuevo, a la caída instantánea ni al wall kick.	Las líneas no sumaban puntos y las nuevas mecánicas no estaban disponibles.	Procedimiento nuc_finalizar_jugada, llamado una vez por jugada. Línea completa: PTS 100 y LN 1.
INT-04	La rama de lógica partía de la base del 28 de octubre con versiones antiguas de los demás módulos.	Una fusión completa habría revertido el trabajo del núcleo y de los gráficos.	Integración selectiva de logica.asm, piezas.asm y andrey/; el main final coincide con la versión probada.
LOG-01	Sobreescritura de registros: clear_top_row modificaba $s0.	El llamador perdía el índice de fila y borraba filas incorrectas.	Se usa $t2 como puntero; $s0–$s7 y $sp se conservan.
LOG-02	update_high_score modificaba $s0 y $s1 sin guardarlos.	Valores alterados en el procedimiento llamador al terminar la partida.	Comparación con $t2 y $t3. Casos menor, igual y mayor aprobados.
LOG-03	No se rechazaban filas negativas antes de calcular la dirección de la celda.	Lectura de memoria anterior a PLAYFIELD y colisiones falsas.	Validación de ambos límites de X e Y antes de cada acceso.
LOG-04	Dimensiones fijas de 16 × 32 en los recorridos.	Recorridos fuera del tablero de 10 × 20 y desbordes de memoria.	Uso de las constantes ANCHO y ALTO en los seis procedimientos.
LOG-05	La colisión solo reconocía el valor 1 como celda ocupada.	Las piezas atravesaban bloques fijados con identificadores 2–7.	Cualquier valor distinto de cero se considera ocupado.

ID	Error detectado	Impacto	Solución aplicada
LOG-06	freeze_piece escribía sin validar la posición.	Posible escritura fuera del tablero que corrompería datos vecinos.	Validación completa de la forma y del tipo antes de escribir.
LOG-07	shift_rows_down no validaba la fila recibida.	Lectura de la fila −1 al desplazar.	Validación de la fila y limpieza directa de la fila 0.
LOG-08	Puntaje de 1 punto por línea calculado dentro del recorrido.	Puntuación incorrecta respecto al esquema clásico.	Conteo por jugada y log_update_score (100/300/500/800 × nivel).
PRU-01	El programa de prueba heredado limpiaba 512 palabras.	Escritura fuera del tablero de 800 bytes durante las pruebas.	Comprobación del tamaño reservado antes de escribir.
XJ-01	draw_scene usaba $s0–$s2 sin guardarlos.	Valores alterados en el llamador tras cada redibujo.	Macros guardar_ra_s0_s1 y restaurar_ra_s0_s1.
XJ-02	El dibujo recorría 512 celdas.	Se pintaban posiciones fuera del tablero de 10 × 20.	Recorrido limitado a ANCHO × ALTO.
XJ-03	Todos los bloques fijados usaban COLOR_FROZEN_BLOCK.	No se distinguía el tipo de pieza fijada.	Color tomado de ALL_COLORS_TABLE[ID − 1]. Verificado en MARS.
G-01	Con el doble búfer, el menú se escribía en video una sola vez, cuando MARS limpia la ventana del Bitmap Display.	Pantalla negra al iniciar; el juego parecía no cargar.	gfx_presentar copia completos los primeros diez cuadros (gfx_cuadros_completos).
E-01	El Keyboard MMIO de MARS no entrega las teclas de flecha; la base leía el teclado con $k0/$k1.	No era posible usar flechas; riesgo de conflicto con registros del kernel.	Se mantienen W, A, S y D; lectura con $t8/$t9 en nuc_leer_tecla.
E-02	El Keyboard MMIO guarda una sola tecla por lectura.	Teclas pulsadas con menos de 100 ms de separación se pierden.	Limitación del simulador; se pulsan las teclas de una en una.
E-03	MARS no recarga un archivo modificado fuera del editor.	Se ensamblaba una versión anterior del código.	Cerrar la pestaña sin guardar y reabrir el archivo antes de ensamblar.

5. Conclusiones

El proyecto permitió aplicar de forma práctica los conceptos de arquitectura MIPS estudiados en el curso: organización de memoria, entrada y salida mapeada en memoria, convenciones de llamada y manejo de la pila. La reorganización de la base en módulos con interfaces documentadas hizo posible que tres personas trabajaran en paralelo, y la integración final confirmó la importancia de definir contratos claros entre procedimientos, como el número de líneas devuelto por check_and_clear_lines o la regla de cerrar cada jugada una sola vez.

Los errores encontrados, en particular los de alineación y preservación de registros, mostraron que en ensamblador los fallos suelen aparecer lejos de su causa, por lo que las pruebas aisladas y la bitácora resultaron herramientas indispensables. El juego final cumple los requisitos del enunciado: estados explícitos, diez niveles con victoria, puntuación clásica, mecánicas completas e interfaz en español sin parpadeo.

Referencias

Patterson, D. A. y Hennessy, J. L. (2014). Computer organization and design: The hardware/software interface (5.ª ed.). Morgan Kaufmann.

Vollmar, K. y Sanderson, P. (2014). MARS: MIPS Assembler and Runtime Simulator (versión 4.5) [Software]. Missouri State University.

tetris_mips_asm (s. f.). Tetris para MARS en ensamblador MIPS [Repositorio de código]. GitHub.